

Conceitos de Linguagens de Programação

Capítulos 1 a 5 de Sebesta

5 capítulos

- 1 fundamentos
- 2 evolução
- 3 descrição
- 4 análise
- 5 variáveis

O que você precisa conseguir fazer

COMPARAR

critérios, paradigmas e estratégias

RELACIONAR

linguagens, domínios e contribuições

APLICAR

gramática, derivação e classificação

EXPLICAR

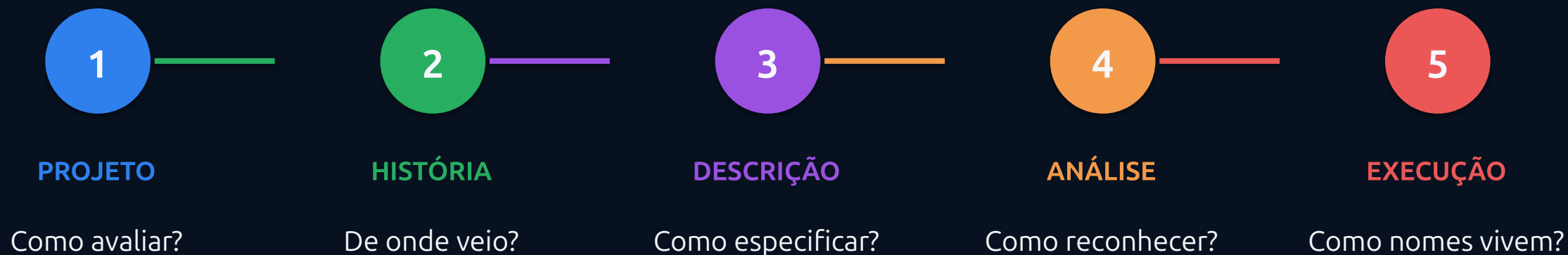
léxico, parser, LR e semântica

ANALISAR

vinculação, vida, escopo e saída

Meta: responder com conceito + justificativa + exemplo.

Uma linguagem passa por cinco lentes



CONEXÃO-CHAVE

A descrição formal orienta o parser; as decisões sobre tipos e memória afetam confiabilidade e custo.

Por que estudar conceitos de linguagens?

EXPRESSAR

ampliar o repertório para formular soluções

ESCOLHER

combinar linguagem e domínio do problema

APRENDER

reconhecer conceitos em novas linguagens

IMPLEMENTAR

entender o que compilador e runtime fazem

USAR MELHOR

explorar com consciência o que já conhece

EVOLUIR

avaliar tendências sem seguir apenas popularidade

Domínio importa: científico ≠ comercial ≠ IA ≠ sistemas ≠ Web.

Escolher exige equilibrar critérios

LEGIBILIDADE

ler e compreender

simplicidade • ortogonalidade •
sintaxe

ESCRITA

expressar soluções

abstração • expressividade •
domínio

CONFIABILIDADE

cumprir especificação

tipos • exceções • poucos aliases

CUSTO

ciclo de vida total

treino • implementação •
execução • manutenção

TRADE-OFF

Mais checagens podem elevar a confiabilidade e também o custo de execução. Mais concisão pode reduzir a legibilidade.

Não existe “melhor linguagem” fora de contexto.

Projeto: influências, categorias e tensões

ARQUITETURA

von Neumann favorece memória, atribuição e iteração — base das linguagens imperativas.

METODOLOGIAS

Estruturada → abstração de dados → orientação a objetos; o projeto de software pressiona o projeto da linguagem.

CATEGORIAS

Imperativa • funcional • lógica • marcação/programação; linguagens reais podem combinar ideias.

confiabilidade



custo de execução

legibilidade



capacidade de escrita

flexibilidade



detecção antecipada

Do código à execução: quatro estratégias

01

COMPILAÇÃO

fonte → máquina

tradução antes da execução

02

INTERPRETAÇÃO

**fonte →
intérprete**

execução instrução a
instrução

03

HÍBRIDA

**fonte →
intermediário**

VM interpreta uma forma
portátil

04

JIT

**intermediário →
máquina**

trechos frequentes
compilados e reutilizados

PISTA DE PROVA

“compila em execução + reutiliza” = JIT

Evolução não é uma fila de substituições



IDEIA CENTRAL

Uma linguagem pode continuar útil em seu domínio e, ao mesmo tempo, transmitir sintaxe, paradigma ou estratégia de implementação a projetos posteriores.

Nova ≠ automaticamente superior • antiga ≠ irrelevante

Quatro linguagens, quatro contribuições



FORTTRAN

computação científica

eficiência numérica e fórmulas



LISP

processamento simbólico

funções, listas e tradição funcional



COBOL

processamento comercial

dados e registros de negócio



SMALLTALK

orientação a objetos

objetos, mensagens e ambiente integrado

A história é uma rede de respostas a domínios — não uma escada única.

Do pioneirismo às famílias híbridas

LINGUAGEM/FAMÍLIA	PROBLEMA/DOMÍNIO	LEITURA PRINCIPAL
Plankalkül	ideias de alto nível	pioneirismo
Fortran / COBOL	científico / comercial	imperativo e domínio
Lisp / Scheme	símbolos e funções	funcional
APL / SNOBOL	arrays / strings	dinamismo e notação
Prolog / Smalltalk	regras / objetos	lógico / OO
Java / Lua	VM-JIT / scripting	estratégias híbridas
JSTL / XSLT	documentos e regras	marcação + programação

Antes da gramática: alinhe o vocabulário

SINTAXE

forma e organização das construções

SEMÂNTICA

significado e efeito das construções

LINGUAGEM

conjunto de sentenças válidas

total = 7;

lexema

ocorrência concreta: total

token

categoria: IDENT

sentença

só terminais

NÃO CONFUNDA

Lexema é o trecho reconhecido; token é a classificação entregue ao parser.

FNB: a anatomia de uma gramática

```
<atr> ::= <id> "=" <expr> ";"  
<expr> ::= <num> | <num> "+" <num>  
<id> ::= "a" | "b"  
<num> ::= "1" | "2"
```

NÃO TERMINAIS

<atr> <id> <expr> <num>

TERMINAIS

a, b, 1, 2, =, +, ;

PRODUÇÃO

define alternativas possíveis

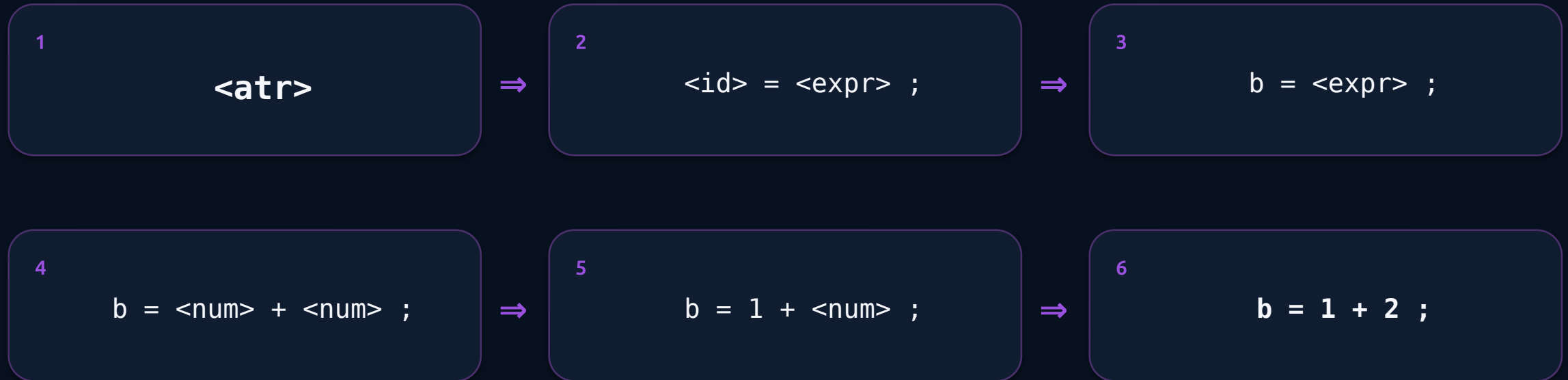
|

significa "ou"

SÍMBOLO INICIAL

ponto de partida: <atr>

Derivação completa: do símbolo à sentença



TESTE DE CHEGADA

Forma sentencial pode conter não terminais. Sentença é uma forma composta apenas por terminais.

Árvore, ambiguidade e FNB estendida

Ambiguidade

uma sentença
→ duas árvores
→ dois agrupamentos

1 - 2 / 3

$(1-2)/3 \neq 1-(2/3)$

FNBE compacta padrões

<code>[item]</code>	opcional
<code>{ item }</code>	zero ou mais repetições
<code>(a b)</code>	agrupamento e alternativa

DESAMBIGUAR

Organize níveis como `<expr>` e `<termo>` para representar precedência e associatividade.

Três maneiras de descrever significado

OPERACIONAL

execução

Descreve mudanças em uma máquina abstrata ou real.

DENOTACIONAL

função matemática

Mapeia construções para objetos e funções matemáticas.

AXIOMÁTICA

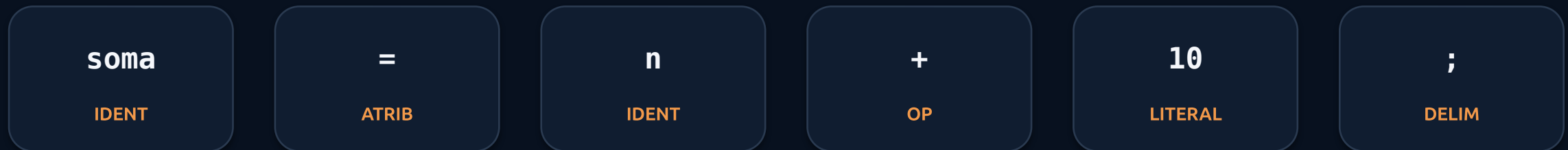
propriedades

Usa asserções, pré/pós-condições e regras de inferência.

OUTRA DIVISÃO ÚTIL

Semântica estática: restrições verificáveis sem executar. Semântica dinâmica: significado durante a execução.

Léxico e parser: cooperação, não duplicação

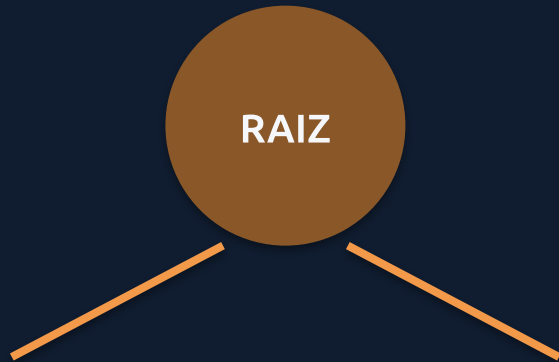


DIVISÃO DE TRABALHO

O léxico reduz detalhes de caracteres. O parser verifica se a sequência de tokens pertence à estrutura da gramática.

Descendente × ascendente

DESCENDENTE



expande produções

derivação mais à esquerda

ASCENDENTE



reduz até o inicial

inverso da derivação à direita

Decidir a regra sem cair em recursão infinita

LOOKAHEAD

O próximo token ajuda o parser descendente a escolher qual produção aplicar.

se token = id → escolha A

RECURSÃO À ESQUERDA

$$E \rightarrow E + T \mid T$$

E chama E antes de consumir entrada
→ recursão infinita no parser recursivo

Reescrita

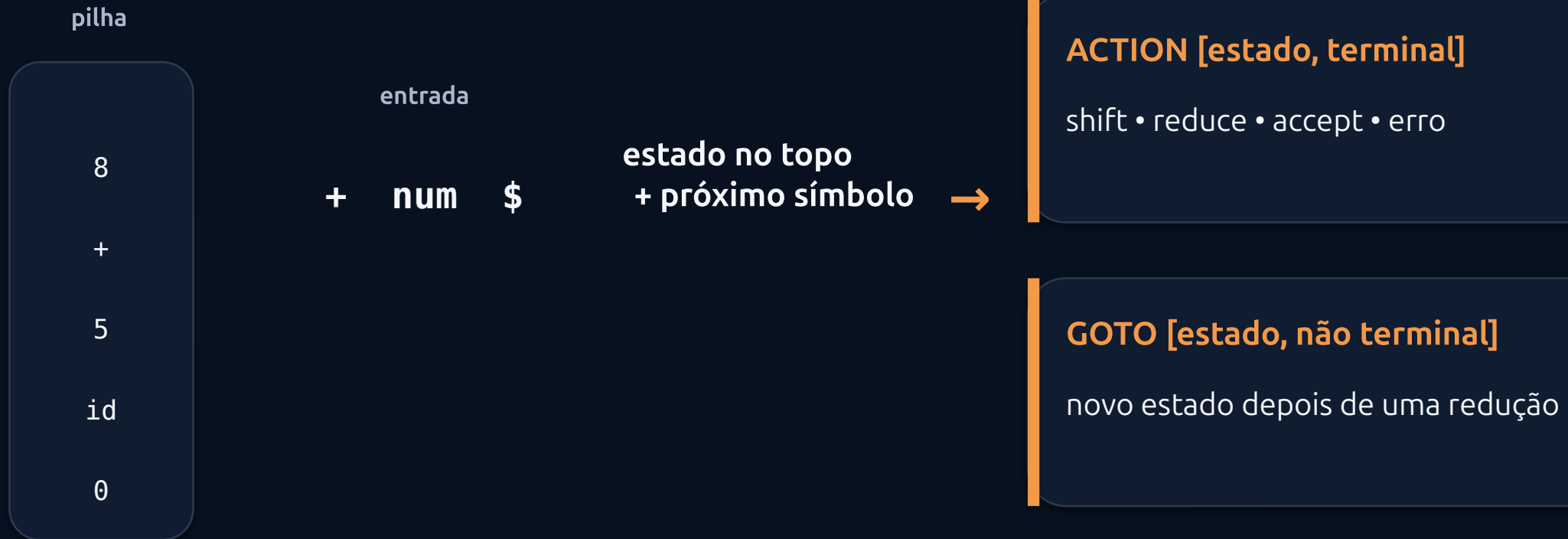
$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \epsilon \end{aligned}$$

consome T primeiro

REGRA PRÁTICA

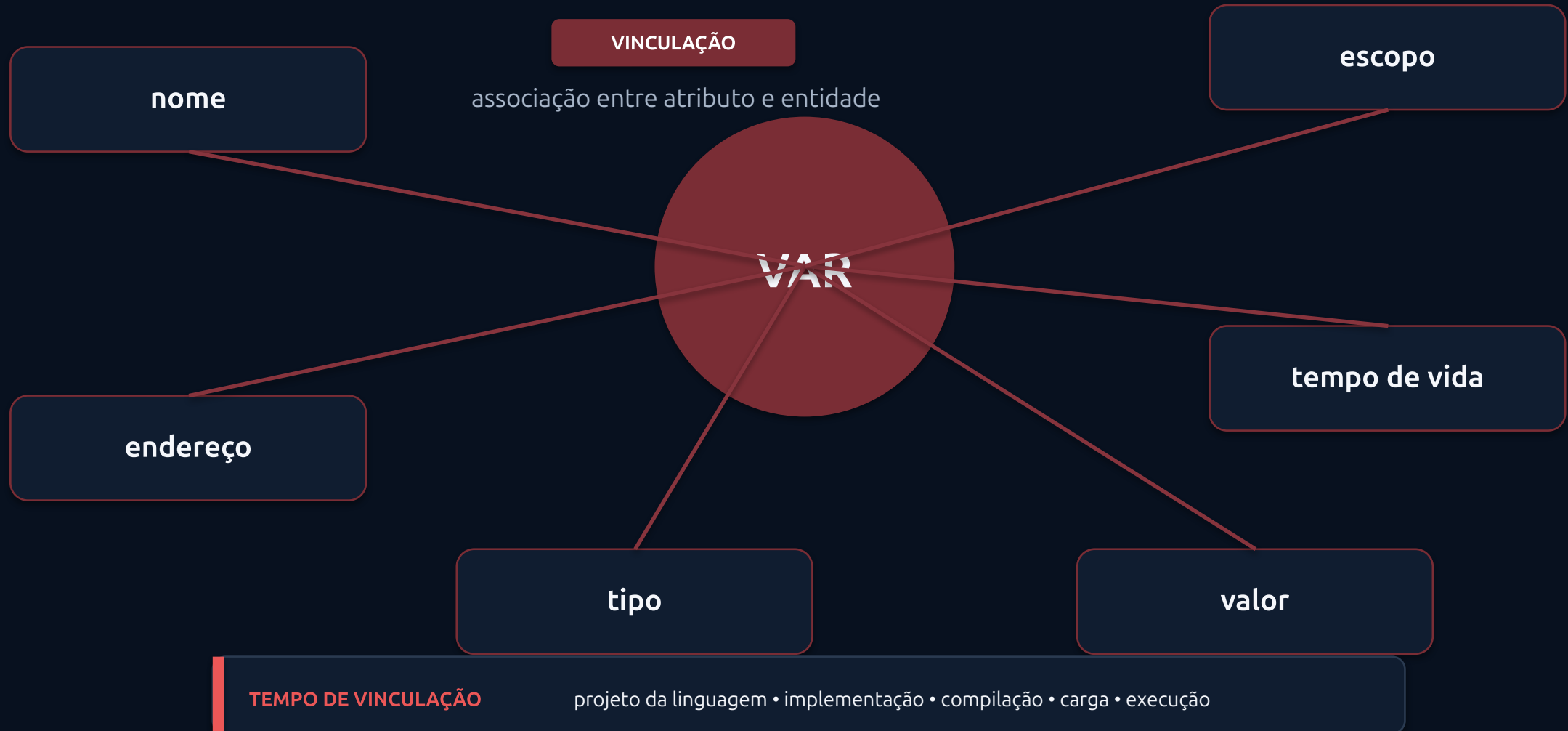
Gramáticas para análise descendente devem ser ajustadas para eliminar recursão à esquerda.

Parser LR: estado + símbolo = decisão



ACTION decide a ação. GOTO conduz a estrutura reduzida.

Variável = entidade + seis atributos



Categorias por tempo de vida e armazenamento

CATEGORIA	ALOCÇÃO / VIDA	EXEMPLO	TROCA PRINCIPAL
estática	antes da execução; vida inteira	static	eficiência ↔ flexibilidade
dinâmica da pilha	entrada do bloco; até o retorno	variável local	recursão ↔ overhead
heap explícita	new/delete ou referência	objeto dinâmico	estrutura ↔ complexidade
heap implícita	atribuições controlam	variável dinâmica	flexibilidade ↔ detecção

CASO-CHAVE

Local comum = dinâmica da pilha. O endereço pode mudar porque cada ativação recebe um novo registro de ativação.

Escopo responde “qual declaração este nome encontra?”

ESCOPO ESTÁTICO

estrutura textual do programa

local → pai léxico → ancestrais

ESCOPO DINÂMICO

sequência de chamadas

local → chamador ativo → cadeia

AMBIENTE DE REFERENCIAMENTO

Conjunto de todas as variáveis visíveis em uma sentença segundo a regra de escopo.

ESCOPO ≠ TEMPO DE VIDA

Escopo trata visibilidade; tempo de vida trata vínculo com uma posição de memória.

Ocultamento: a declaração mais próxima pode esconder outra com o mesmo nome.

Escopo estático e ocultamento em ação

```
class Paine1 {  
    static int nivel = 8;  
    static void exibir() {  
        int nivel = 3;  
        System.out.print(nivel + " " );  
    }  
    public static void main(String[] a) {  
        exibir();  
        System.out.println(nivel);  
    }  
}
```

PARE E PREVEJA

saída

3 8

DENTRO DE exibir

o nome local nivel oculta o atributo da classe

DE VOLTA À main

a busca encontra o atributo estático nivel

Se você consegue explicar isto, cobriu a prova

1

Critérios + trade-off
JIT e implementação

2

Fortran • Lisp • COBOL •
Smalltalk
evolução como coexistência

3

derivação e terminais
ambiguidade e 3 semânticas

4

lexema/token +
léxico/parser
descendente/ascendente +
LR

5

pilha, vinculação e vida
escopo estático +
ocultamento

Resposta forte = termo correto + mecanismo + consequência.

Checklist final de 10 minutos



Defino os quatro critérios sem confundi-los.



Relaciono quatro linguagens a seus domínios/contribuições.



Distingo três abordagens semânticas e ambiguidade.



Comparo descendente × ascendente e explico LR.



Separo escopo de tempo de vida.



Reconheço JIT pela compilação durante a execução.



Faço uma derivação e separo terminal de não terminal.



Classifico lexemas e explico léxico × parser.



Classifico variável local como dinâmica da pilha.



Prevejo saída com ocultamento no escopo estático.

Explique em voz alta. Onde travar, volte ao slide do capítulo.

Fontes completas e localizações estão nas notas do apresentador e no arquivo .meta.json.